

# Lexique et morphologie

13 octobre 2025

—Kata Gábor

*kata.gabor@inalco.fr*—

## Morphologie computationnelle : définition "intuitive"

- **analyse morphologique** : étant donné une forme, y associer la catégorie grammaticale, les catégories flexionnelles et le lemme

| entrée :<br>forme de mot | sortie :<br>lemme + info morphologique |                   |
|--------------------------|----------------------------------------|-------------------|
| dit                      | dire                                   | V +sg +3 + pres   |
| dit                      | dire                                   | V +ppassé         |
| dit                      | dire                                   | V +sg +3 +psimple |

- **génération** morphologique : étant donné un lemme et des catégories flexionnelles, trouver la forme correspondante

| entrée :<br>lemme + info morphologique |                   | sortie :<br>forme de mot |
|----------------------------------------|-------------------|--------------------------|
| parler                                 | V +sg +3 + pres   | parle                    |
| parler                                 | V +ppassé         | parlé                    |
| parler                                 | V +sg +3 +psimple | parla                    |

**Tâches connexes** : tokenisation, normalisation, reconnaissance des entités nommées (avant) ; lemmatisation (au lieu de), étiquetage morphologique ou POS tagging (après)

**Applications** : correcteurs d'orthographe, moteurs de recherche, traduction automatique : analyse et génération, systèmes d'interaction homme-machine, reconnaissance de la parole

## Morphologie computationnelle : définition formelle

L'analyse morphologique (M) consiste à définir une relation sur deux ensembles :

1. l'ensemble des formes de mots (p.ex.  $F = \{\text{aller, va, ira, allé}\}$ )
2. l'ensemble des descriptions : (p.ex.  $C = \{\text{aller+V+inf, aller+V+sg+3, ...}\}$ )

$$M \subseteq F \times C \quad (1)$$

où  $F \times C$  dénote le produit cartésien, c'est-à-dire l'ensemble des paires possibles entre les formes et les descriptions :

$$F \times C = \{(f, c) | f \in F \text{ et } c \in C\} \quad (2)$$

## Stratégies pour l'analyse morphologique

1. reconnaissance des formes sans règles : l'approche "**lookup**".  
Il s'agit d'énumérer toutes les formes possibles, éventuellement en y associant les analyses morphologiques, dans un dictionnaire. L'analyse morphologique consiste à chercher la forme du mot dans ce dictionnaire pendant le temps d'exécution. Les formes prédictibles y sont traitées de la même manière que toute information non prédictible.
2. les approches **génératives** : l'analyse tient compte des régularités ; cette approche permet de rajouter de nouveaux mots sans avoir à préciser toutes ses formes (on génère les différentes formes régulières/prédictibles associées au même mot)
  - (a) les approches **morphémiques** : les formes prédictibles sont générées en décomposant les formes complexes en morphèmes. Permet de rajouter de nouveaux mots et générer toutes leurs formes prédictibles et segmentables ; difficultés pour les formes prédictibles mais non segmentables, p.ex. *ablaut*.
    - **automates à états finis** pour la reconnaissance des formes
    - **transducteurs à états finis** pour remplacer les formes par des analyses (lemme, catégorie grammaticale, informations flexionnelles)
  - (b) les approches **implicatives** (*mot et paradigme/word and paradigm*) : au lieu de segmenter en morphèmes, les formes sont produites par des opérations comme la suppression, l'addition, ou la substitution de lettres. Des relations implicatives sont définies entre les formes complexes. P.ex. : si le latin **rosa** forme son génitif singulier en *rosae*, son accusatif du pluriel sera produit par substitution de 's' à la place du 'e' final : *rosas*.

## I. L'approche "lookup"

- stocker *toutes* les formes dans un dictionnaire
- lire l'entrée et rechercher dans le dictionnaire en temps d'exécution
- contre : ignore les régularités dans la morphologie, l'ajout de nouveaux mots nécessite d'ajouter toutes leurs formes
- pré-requis : *générer* toutes les formes (!) → largement possible pour l'anglais ou le français ; très problématique pour le turc, l'arabe, l'allemand, le hongrois etc

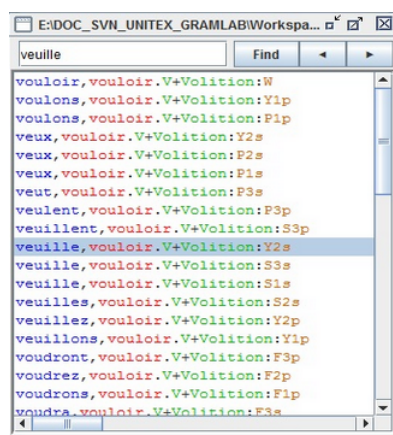


FIGURE 1 – Dictionnaire pour la méthode lookup : formes et analyses

outil d'analyse avec méthode lookup : **automate fini déterministe acyclique** (DAFSA, Deterministic Acyclic Finite State Automaton) :

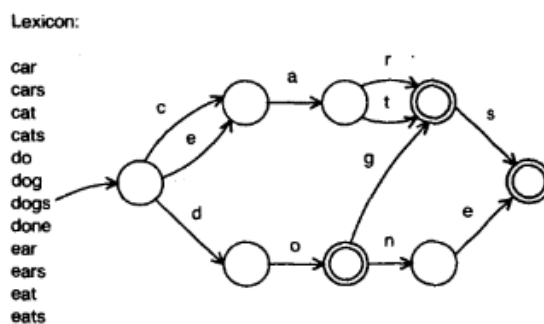


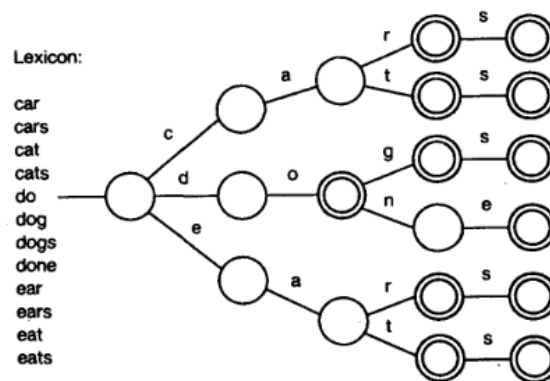
FIGURE 2. A Dawg

FIGURE 2 – Lexique et deterministic acyclic finite state automaton (DAFSA) correspondant

- un seul état d'entrée
- chaque transition est étiquetée par un symbole
- déterministe : une seule transition par étiquette
- acyclique / fini : le nombre des noeuds et des transitions (arêtes) est fini et ne contient pas de cycle : pas de retour à un noeud déjà visité
- possible de tester si une chaîne de caractères est reconnue par le DAFSA dans un temps proportionnel à la longueur de la chaîne

**référence : Appel and Jacobson (1998) : The World's Fastest Scrabble Program.**

un cas spécial des DAFSA : les *arbres de recherche* (un DAFSA où un état ne peut être atteint de deux manières différentes)



**FIGURE 1. A Lexicon and the Corresponding Trie (Terminal Nodes are Circled)**

**FIGURE 3 – Lexique et arbre de recherche correspondant**

## Automates à états finis

Les automates à états finis reconnaissent un certain type de langues : les langues régulières.

### Langue régulière :

$L$  est une langue régulière sur l'alphabet  $V$  :

- $\emptyset$  (l'ensemble vide/langage vide) est une langue régulière ;
- $\forall a \in V, \{a\}$  (l'ensemble composé d'un élément du vocabulaire) est une langue régulière ;
- Si  $L_1$  et  $L_2$  sont des langues régulières,
  - $L_1 \cup L_2$ , l'**union** de  $L_1$  et  $L_2$ ,
  - $L_1 \cdot L_2$ , la **concaténation** de  $L_1$  et  $L_2$ , c'est-à-dire  $\{xy \mid x \in L_1, y \in L_2\}$
  - $L_1^*$ , la **fermeture de Kleene** de  $L_1$ , c'est-à-dire la langue obtenue en prenant un nombre quelconque (y compris 0) de chaînes de  $L_1$  et en les concaténant.

Un **automate à états finis** est constitué de :

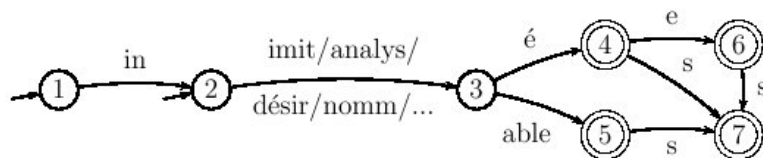
- $V$ , un vocabulaire ou alphabet **fini**
- $Q$ , un nombre fini d'états dont
  - état initial (un ou plusieurs)
  - état final (un ou plusieurs)
- $f$ , fonction de transition : donne pour chaque état  $q \in Q$  et chaque élément du vocabulaire  $v \in V$ , le ou les états que l'on peut atteindre en partant de  $q$  et en utilisant  $v$  :  $f(q,v) \subset Q$ .

P.ex. : automate permettant d'analyser les mots comme : "*inimitables*", "*imitée*", "*analysées*", "*inanalysable*", "*désirable*", "*indésiré*", "*innomable*"

$V = \{in, imit, désir, analys, nomm, able, é, e, s\}$

- racines *imit*, *désir*, *analys*, *nomm*
- préfixe *in*
- suffixe *able*, *é*, *e*, *s*

visualisation de l'automate par graphe :



- ici, les états sont représentés par des noeuds numérotés 1-7

- états initiaux : ceux sur lesquels arrive une flèche qui vient de nulle part : ici, 1 et 2
- états finaux : 4, 5, 6, 7 (double rond par convention)
- transitions : flèches/liens entre les états avec étiquette de transition (élément du vocabulaire), p ex  $f(3, \text{able})=5$
- les boucles (transition d'un état vers lui-même) sont en principe autorisées

Un **chemin** dans un automate commence nécessairement à partir d'un état initial, suit des transitions et aboutit à un état final.

Le **langage** accepté/défini par l'automate est l'ensemble de concaténations d'éléments du vocabulaire  $V$  qui correspondent à un chemin.

Il existe d'autres automates reconnaissant le même langage. Par exemple, un automate avec seulement un état initial et un état final, et autant de transitions que de mots distincts conviendrait tout aussi bien. Mais celui de la figure met en évidence des régularités de construction que n'auraient pas l'autre (équivalent à une simple liste).

**Avantages des automates :**

- bien adaptés pour modéliser l’affixation :
  - conjugaison en français
  - déclinaisons dans les langues à cas
  - dérivations productives
- rapidité
- facile à combiner par des opérations d’union, concaténation, ...

Les automates finis reconnaissent une classe de langages particulière appelée langages réguliers ou langages rationnels.

**Implémentations**

- KIMMO, PC-KIMMO : Koskenniemi 1983, Karttunen 1983
- pour la vitesse de l’analyse :
  - XFST : Xerox Finite State Tool
  - SFST : Stuttgart Finite State Tool (morphologie anglaise, allemande, turque, latine disponible)
- pour la facilité et l’interface graphique :
  - Unitex, [https ://unitexgramlab.org/fr](https://unitexgramlab.org/fr)
  - NooJ, [http ://www.nooj-association.org/](http://www.nooj-association.org/)

## Exercices

1. Construisez un automate morphologique qui reconnait

(a) les déclinaisons latines suivantes :

| Cas       | Singulier | Pluriel |
|-----------|-----------|---------|
| Nominatif | Rosa      | Rosae   |
| Accusatif | Rosam     | Rosas   |
| Génitif   | Rosae     | Rosarum |
| Datif     | Rosae     | Rosis   |
| Ablatif   | Rosa      | Rosis   |

| Cas       | Singulier | Pluriel |
|-----------|-----------|---------|
| Nominatif | Murus     | Muri    |
| Accusatif | Murum     | Muros   |
| Génitif   | Muri      | Murorum |
| Datif     | Muro      | Muris   |
| Ablatif   | Muro      | Muris   |

(b) la conjugaison des verbes français en -er : à l'indicatif présent, imparfait, futur